

Phase 2 — Foundations & Contracts

LineTwin · Accenture Innovation Challenge 2026 · Round 2 · Problem Track 4 "DigitalTwin.ai"

Purpose

Establish the repository, verify every inherited dependency pin against reality rather than trusting it, decide the transport mechanism by spiking it rather than guessing, and freeze the data contract every later phase — including Phase 9's defect genealogy — will code against.

What was verified, not assumed

Two checks in this phase existed specifically to convert an inherited assertion into a fact.

1. Dependency pins. The build brief this project inherited asserted eight version pins as "verified against PyPI JSON." A first attempt to check this with `uv pip download` failed for all eight identically — which was itself a signal the *check* was broken, not the pins (`uv pip` has no `download` subcommand). Re-verified against the PyPI JSON API directly: all eight pins (`simpy==4.1.2`, `fastapi==0.141.1`, `uvicorn==0.52.4`, `pydantic==2.13.4`, `numpy==2.5.2`, `scikit-learn==1.9.0`, `xgboost==3.4.1`, `sse-starlette==3.4.8`) are in fact each package's current latest release. `uv sync --all-extras` then resolved the full dependency graph including the `ml` and `dev` groups without conflict, on a system Python of 3.13 provisioning the pinned 3.12 interpreter itself.

2. PDF generation toolchain. An earlier document assumed a converter existed for the ten requested phase PDFs. Checked directly: no LaTeX (`pdflatex` / `xelatex` / `lualatex` / `tectonic`), no `weasyprint`, no `wkhtmltopdf`, no `prince`; `cupsfilter` produces no output on HTML input. **Chrome headless --print-to-pdf was tested and confirmed working** — a 13.8 KB valid one-page PDF from a minimal HTML file. `tools/mkpdf.sh` wraps this: markdown → styled HTML (Accenture palette) → Chrome headless → PDF.

The SSE spike — and what it found

The plan carried an unresolved decision: two documents disagreed on whether the transport should be `sse-starlette==3.4.8` (pinned as a dependency) or `fastapi.sse` (claimed to be "documented, added in FastAPI 0.135.0"). Rather than picking one on paper, both were spiked against a running server before `pyproject.toml` was finalized.

Decision: `fastapi.sse`, built into FastAPI 0.141.1. `sse-starlette` dropped entirely — one fewer runtime dependency for no loss of capability.

The spike surfaced two failure modes that would not have been obvious from reading the import surface, recorded in full in `docs/adr/ADR-002-transport.md` :

- **EventSourceResponse is a marker, not a wrapper.** The first spike attempt returned `EventSourceResponse(gen())` from an ordinary route function. It produced a healthy `200 OK` with the correct `text/event-stream` content-type and then delivered **zero frames** — a server crash inside the response encoder (`AttributeError: 'ServerSentEvent' object has no attribute 'encode'`), the worst possible failure signature to debug against a live dashboard later. The correct pattern is `response_class=EventSourceResponse` on a route that is itself the async generator.
- **data= double-encodes pre-serialized JSON.** `ServerSentEvent` carries both `data: Any` (serialized by the routing layer) and `raw_data: str | None` (passed through verbatim). Passing `json.dumps(...)` to `data=` produces a JSON string containing JSON — a browser's `JSON.parse(event.data)` would return a string, not an object, requiring a second parse.

Corrected spike, re-run and verified: correct headers (`content-type: text/event-stream; charset=utf-8, cache-control: no-cache`), 25 `snapshot` frames delivered in 3 seconds against an 8 Hz ticker, `seq` strictly increasing, clean client disconnect with no traceback, and a direct assertion that `JSON.parse(event.data)` yields an object rather than a string.

Deliverables produced

Artefact	What it fixes
<code>pyproject.toml</code>	Pinned, PyPI-verified deps; <code>ml / dev</code> groups isolated from the server import path; <code>scipy</code> added explicitly ahead of Phase 5's significance testing
<code>docs/adr/ADR-002-transport.md</code>	The SSE decision and both non-obvious failure modes, so Phase 6 does not rediscover them
<code>src/twin/contracts.py</code>	Frozen pydantic schema: <code>Snapshot</code> , <code>StationSnapshot</code> , <code>BottleneckVerdict</code> , <code>TaggedValue</code> (<code>OBSERVED/INFERRED/SIMULATED</code>), <code>Missingness</code> (<code>ZERO/MISSING/NOT_APPLICABLE</code>), and the <code>UnitEvent</code> schema Phase 9's genealogy will consume unchanged
<code>src/twin/provenance.py</code>	Machine-readable Kritzing/Grievous/Villegas classification — <code>describe()</code> returns the same claim the README states, so it is a runtime fact, not only prose
<code>src/twin/sources.py</code>	The <code>TelemetrySource</code> ABC (read-only by construction — no <code>write</code> method exists) and <code>ReplaySource</code> , a fixture-backed implementation with zero <code>simpy</code> dependency
<code>tools/gen_fixture.py</code>	Generates a schema-conformant 60-tick, 30-station fixture rather than hand-authoring thousands of values

Artefact	What it fixes
<code>fixtures/replay_30x60.json</code>	The generated fixture: 22 of 30 stations instrumented, matching <code>docs/DECISIONS.md</code> 's committed split
<code>tests/test_fixture_matches_contract.py</code>	8 tests: schema conformance, tick monotonicity, the exact <code>sim_time_s == tick * SIM_DT</code> invariant, the instrumentation split, and — the single most load-bearing fact in the diagnostic layer — that <code>StationState.DOWN.is_active</code> is <code>True</code> and <code>STARVED / BLOCKED</code> are <code>False</code>
<code>tests/test_source_agnostic.py</code>	4 tests proving the source layer is source-agnostic: <code>simpy</code> is asserted absent from <code>sys.modules</code> throughout, which cannot be faked by a script
<code>.github/workflows/ci.yml</code>	ubuntu + windows matrix, authored and YAML-validated; not yet run — see below

What is deliberately not yet true

CI is authored, not green. GitHub Actions requires a push, and the plan's own rule — no remote push without explicit sign-off — was not lifted this phase. Making "CI green" a Phase 2 exit criterion while simultaneously forbidding the action that would produce it is a contradiction, not a gate; it is deferred to Phase 10 rather than silently ignored. The workflow file is validated for YAML syntax now.

Windows is untested locally. This machine is darwin/arm64. The workflow's Windows job exists to surface a wheel-resolution failure early once pushed, and is explicitly non-blocking for the local demo per `docs/DECISIONS.md` — ubuntu is the deploy target.

A dependency was found and removed during this phase

`ruff` flagged `tools/gen_fixture.py` 's `sys.path.insert(...)` + `# noqa: E402` as dead weight. Checked directly rather than assumed: because the project is installed editable via `uv sync`, `twin.contracts` is importable from any working directory without the path manipulation. Removed both the `sys.path` hack and the `noqa`, and switched `StationState / ValueSource / Missingness / Zone` and the three provenance enums from `(str, Enum)` to `enum.StrEnum` on `ruff` 's `UP042` suggestion — functionally identical serialization, less code. `ruff check` . is clean.

Exit criteria

Criterion	Status
<code>uv lock / uv sync</code> resolves the full dependency graph	Met — all 8 inherited pins verified against PyPI, <code>ml / dev</code> groups isolated
Fixture conforms to the frozen contract	Met — 8/8 tests green
SSE mechanism decided and recorded, not left ambiguous	Met — <code>fastapi.sse</code> , ADR-002, two failure modes documented
CI workflow authored and syntactically valid	Met — YAML validated; run deferred to Phase 10 pending push sign-off
Source-agnosticism proven, not asserted	Met — 4/4 tests green, <code>simpy</code> absent from <code>sys.modules</code> throughout
Lint clean	Met — <code>ruff check .</code> passes with zero errors

12/12 tests passing.

Next

Phase 3 — Simulation Core. Parameter grounding against Future Factories V2 and PyScrew; the `lognormal_params` helper as the sole caller of `rng.lognormal`; per-station seeded generators; the merged station pattern carrying both load-bearing guards (Bug A: downstream `put` while still occupied; Bug B: the `.triggered` check before STARVED/BLOCKED); 30 stations across 3 zones with per-unit event logging against the schema frozen this phase. Exit gate: both regression tests green *and* proven to fail red when their guard is removed.